q1

September 25, 2026

[34]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[35]:
```
df = pd.read_csv('sampregdata.csv', index_col = 0)
```

[36]:
```
df.head()
```

[36]:

	x1	x2	x3	x4	y
1	7.331693	9.660958	-2.981869	5.214284	-2.311878
2	16.888609	9.546231	18.440459	11.804018	0.994307
3	8.280643	6.442062	-0.225791	6.493211	-5.253876
4	3.827778	3.741506	-2.850664	-0.226420	20.056798
5	9.675971	4.385008	8.344576	5.979043	9.577914

[37]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = df.drop(columns = ['y'])
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,␣
↪random_state = 1)
```

[38]:
```
features = X_train.columns
min_mse = float('inf')

for feature in features:
    candidate_model = LinearRegression()
    candidate_model.fit(X_train[[feature]], y_train)
    y_pred = candidate_model.predict(X_test[[feature]])
    curr_mse = mean_squared_error(y_test, y_pred)

    if curr_mse < min_mse:
        min_mse = curr_mse
        best_feature = feature
        best_one_feature_model = candidate_model
```

```
print(f"Best feature: {best_feature}, Best MSE: {min_mse}")
```

```
Best feature: x2, Best MSE: 84.4100477060547
```

[39]:
```
features_reduced = X_train.drop(columns = [best_feature]).columns
min_mse = float('inf')

for feature in features_reduced:
    features_to_use = [best_feature, feature]
    candidate_model = LinearRegression()
    candidate_model.fit(X_train[features_to_use], y_train)
    y_pred = candidate_model.predict(X_test[features_to_use])
    curr_mse = mean_squared_error(y_test, y_pred)

    if curr_mse < min_mse:
        min_mse = curr_mse
        best_features = features_to_use
        best_two_feature_model = candidate_model

print(f"Best feature: {best_features}, Best MSE: {min_mse}")
```

```
Best feature: ['x2', 'x4'], Best MSE: 55.63805895191378
```

[40]:
```
# sanity check that the models are accessible
pred_one_feature = best_one_feature_model.predict(X_test[[best_feature]])
pred_two_features = best_two_feature_model.predict(X_test[best_features])

print(np.isnan(pred_one_feature).sum())
print(np.isnan(pred_two_features).sum())
```

```
0
0
```

Q1: https://github.com/dennisous/STT890-MLOps

Q2:

a. What is a demand planner?

Someone who is responsible for estimating future customer demand for products, while maintaining the demand plan for the broader supply chain score. One crucial task is creating and maintaining forecasts, subject to change from statistical analysis or established business knowledge. The created and maintained reports are passed on to supply planning teams.

b. Why would a demand planner need product forecasts?

A demand planner needs product forecasts as a historical baseline for future product manufacturing, purchasing, and inventory specific tasks (how much product to hold, when to replenish inventory and so on). Without using historical baselines, it's much more difficult to understand future product production creation.

c. What time frequency is most likely needed?

According to SAP and Oracle sources, it seems like weekly is the most common. Having daily forecasts for 100 products comes with noise and operation hassle + cost. There are also monthly level demand forecasts, but regardless of the time dimension, the forecasts all seem to be on the product, customer, and location level. I initially assumed it would just be on the product and location level, however a customer difference of a 1000+ customer company vs and individual contractor, for example, clearly would likely have a large effect on a majority of the process steps within the forecast.

d. What data do you need to get started?

The minimum data that I would need to get started is the historical demand + sales for products by location and time period, with product_id keys, date (weekly granularity for now), and quantity ordered/sold. According to AWS, historical demand is almost always the time series target and includes the necessary variables I mentioned above, but also information such as promotion, inventory details, and product category are used as training features. There are also variables that denote the time out before demand prediction on the product level which I didn't initially think of, yet makes total sense.

e. What would I ask the demand planners?

- Time: when do you need to forecast by (weekly, monthly, product specific)?
- What tools or processes do you currently use to forecast demand?
- What level do the forecasts need to be at? Product level only , product and location level product family level, ect.
- Are there certain products that are more difficult to predict demand for?
- What promotions cause the most change/disruption to typical demand forecasting? Seasonality? Initial launches? Marketing campaigns?
- What are the most costly errors in forecasting?

f. How would you show the planners that the models should be trusted?

Since at the current day, the predictions would be focused on future results where evaluation is impossible at the time, we would need to get backtest results for the model. Apply the model during historical periods where forecasted demand is recorded, known and pretended it is not. Evaluate the backtest performance by comparing the model's results to the true demand. It would be beneficial to show predicted vs actual demand on the product level, especially for some of the more difficult products to forecast for. According to the AWS article as well, predictor explainability and quantile forecasts can help with the uncertainty and trustworthiness of the model

g. What characteristics should the production data have?

The production data needs to arrive consistently, automatically and at the same granularity that was defined with the model building population. There should be no duplicate transactions, no missing product ids, consistent timestamps that map to a forecast period such as one week or month ahead, and of course the required model input data before a forecast can be run. The AWS docs emphasize that variables denoting product discounts/sales should be labeled at the proper frequency in order to accurately forecast when such an event may occur in the future.

h. How would they most likely want to receive the results?

Rather than showing raw forecast predictions or dashboards, it would be better to include the predicted forecast within the existing demand forecasters software/tool kit. SAP describes its planning board in a spreadsheet style, meaning columns could denote key information such as historical demand, upcoming weekly or monthly forecasts, forecasting intervals, where the rows pertain to either specific product IDS, potential aggregate product groups based on category. A key detail is that the model forecasting predictions are supposed to influence the decisions the demand forecaster makes, so the ability to deny or accept prediction demand forecasts, as well as review and change forecasts based on the prediction information is a must.

i. Sources used

- SAP Help Portal, to understand the demand planner's role, workflow, and delivery format. SAP Help Portal
- Oracle Supply Chain Planning documentation, demand-forecast configuration documentation, for typical weekly/monthly planning cycles. Oracle Docs
- Amazon Web Services documentation, Retail and Inventory Planning forecasting domains. AWS Documentation

1 oven, 4 burners

Roast + 1 other item

Dependency
Roast beef → gravy
85 min 12 min

Roast beef 100 min total, longest task. Pieces cannot be shorter than this

ANS: 100 minutes

TIME in Minutes

0 10 20 30 40 50 60 70 80 90 100

Roast beef — 0–15 min; 85–100

Gravy — 85–97

Mix and close — 0–20; 50

Glazed carrots — 5–23; 43–48

Salad — 23–40

Dinner roll — 50–58; 73

Mashed potatoes — 20–30; 55–63

Green Beans — 30–48

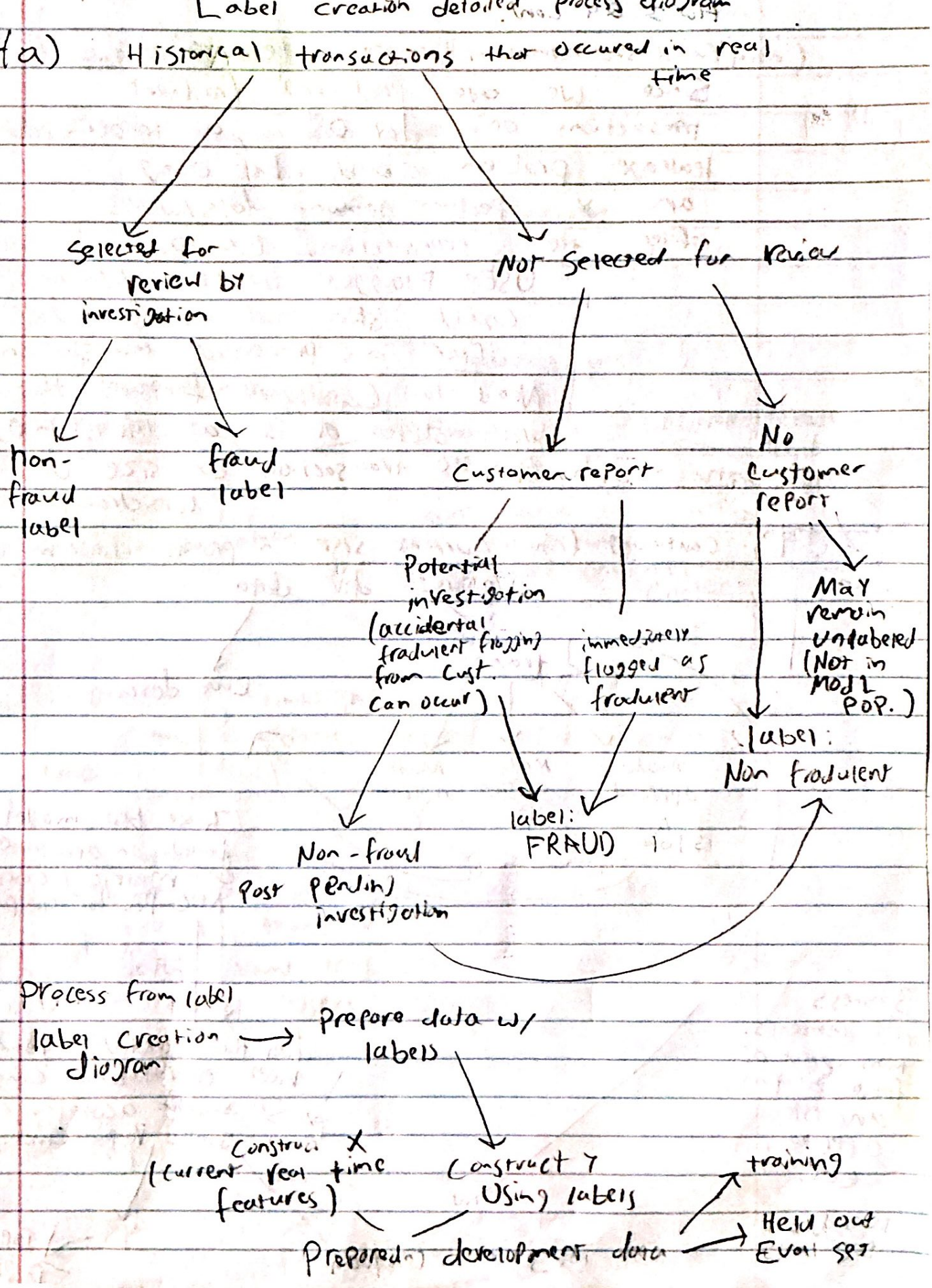
Cook 1

Cook 2

---- Unattended Period

HW 1 Q4

4a) Label creation detailed process diagram



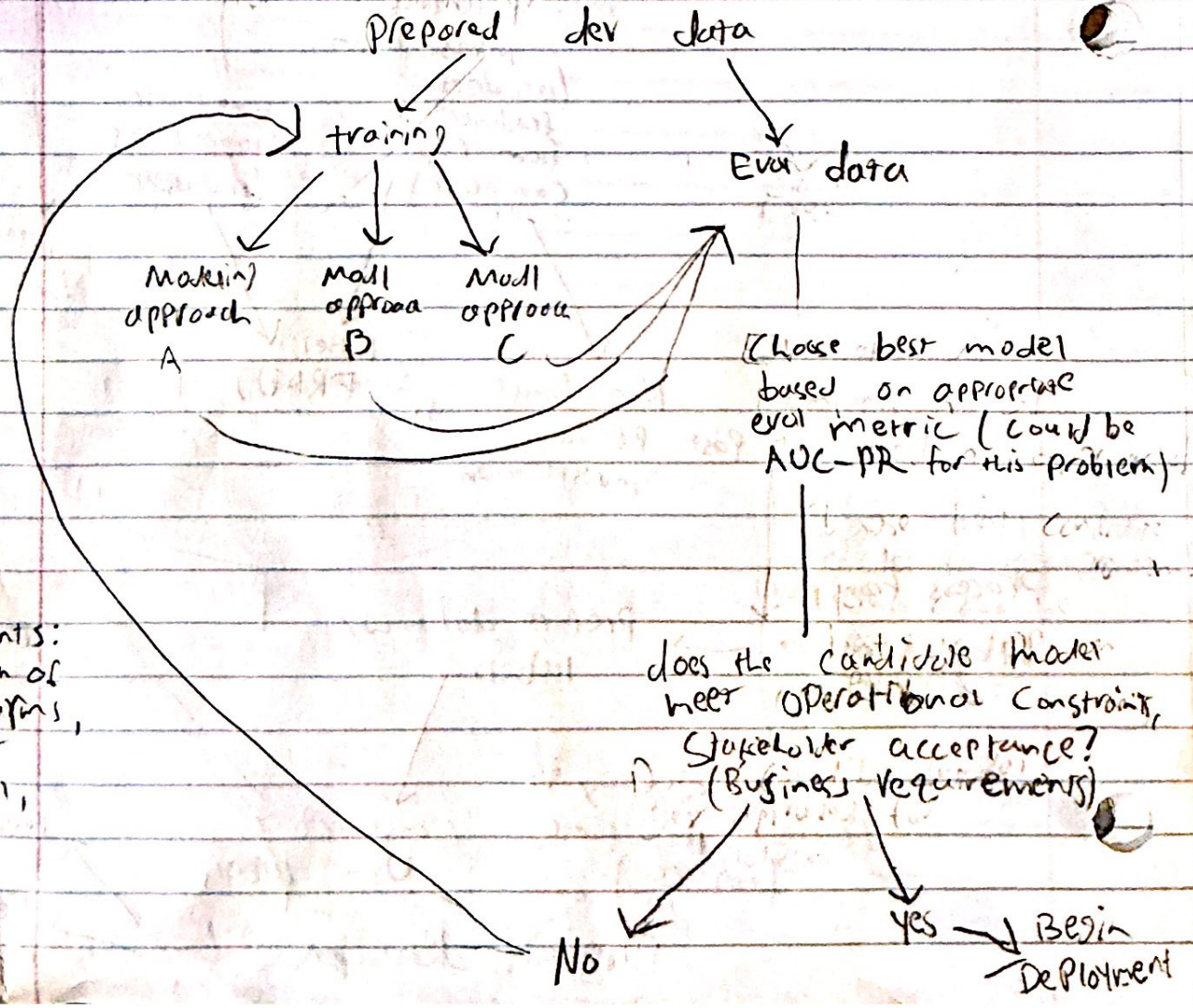
HW 3 Q4 cont.

Problems with target leakage

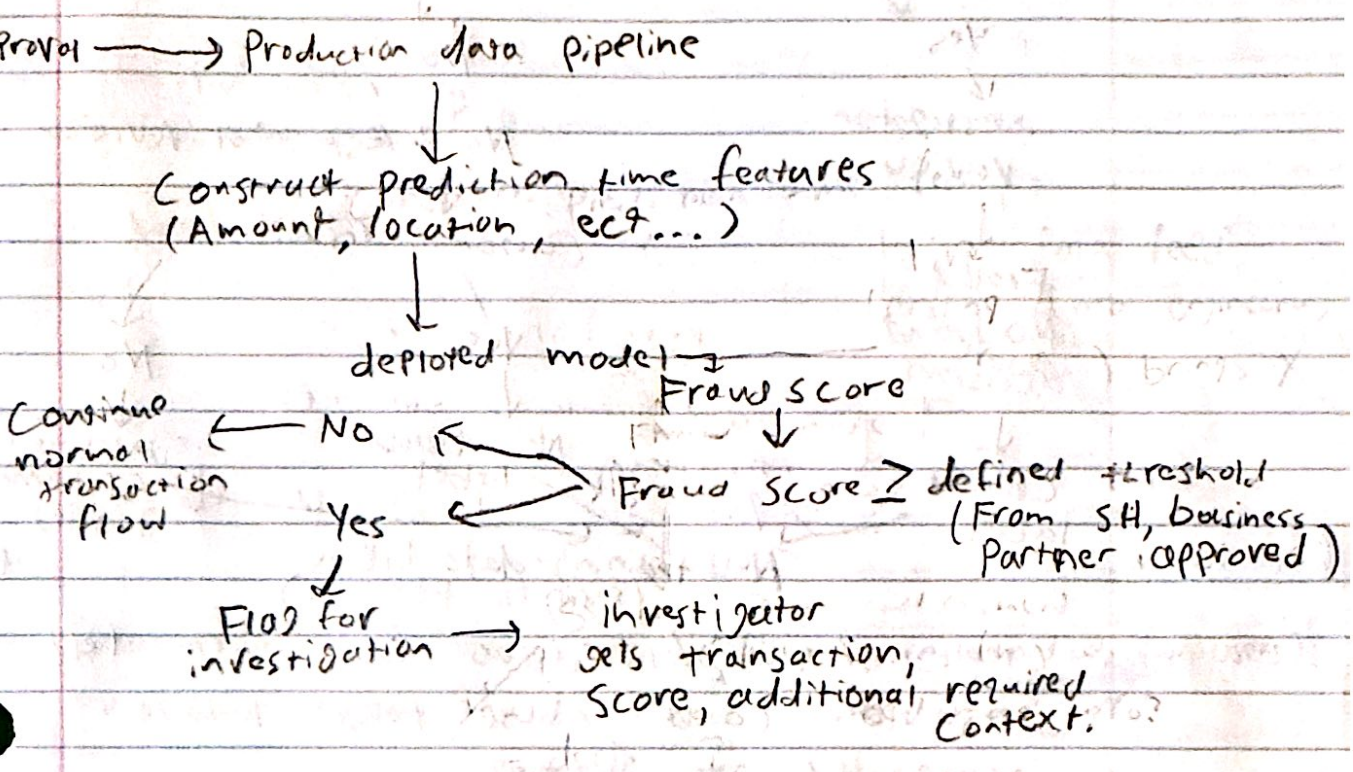
Since we are predicting fraudulent transactions on day 0, a target leakage problem could be using an X feature known days/weeks after the transactions. Example USER_Flagged binary indicator. Could show up as 3 days after a processed transaction.

- Need to configure features that will appear as is at the timing of the transaction, Ex size of withdrawl.

b) Continuing from current step in process diagram.



c) deployment Phase (Model passes all criterion)



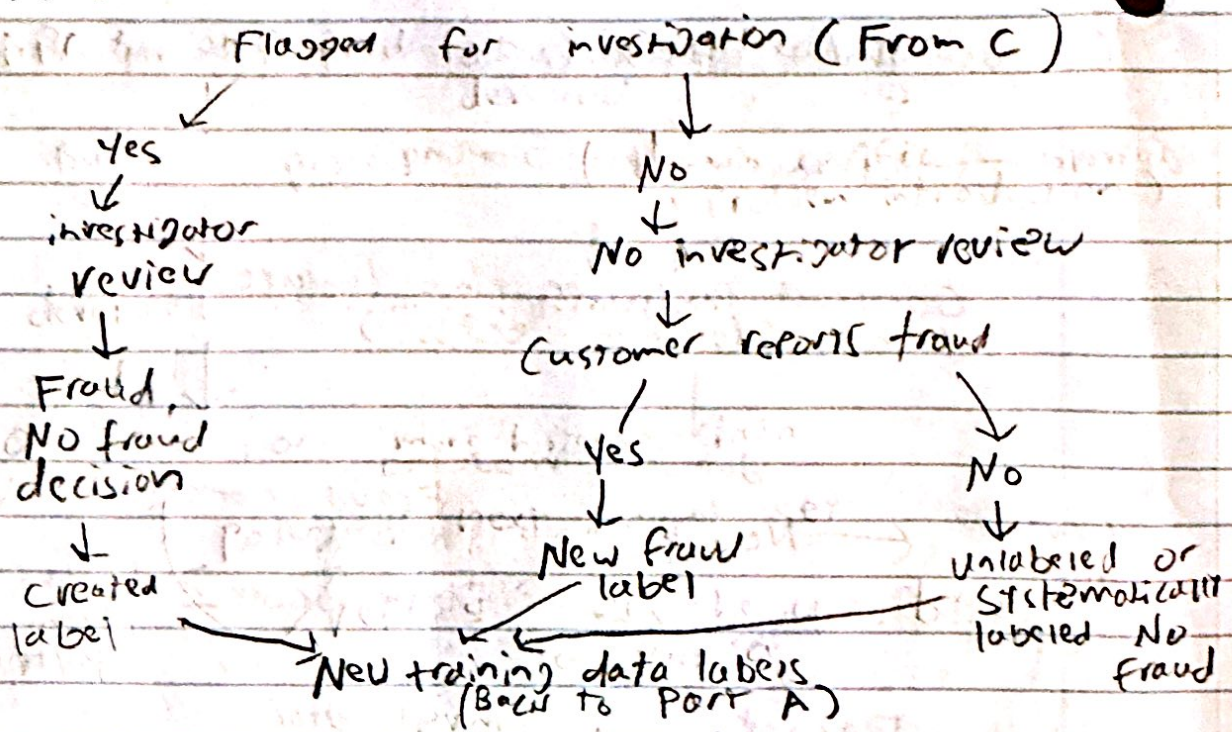
d) prediction - time information
transaction data, derived input features

Time ↓
Model output fraud score

Later information
1. actual outcomes, what actually happened to the
2. flagged transactions?
3. Collect ground truth information. - Decisions from investigators or model users (essentially outcome)

HW 1 Q4 cont.

e)



- Creating variables / indicators for investigated transactions could cause target leakage if incorporated into training

- Since the model influences which cases receive scrutiny, which affects how labels are discovered, then the model may have an influence on the future training data.

q5

September 25, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
```

[2]:
```
n = 1000000
sh_analysis = stats.expon.rvs(scale=10, size=n)
extr = stats.norm.rvs(loc=15, scale=3, size=n)
eda = stats.norm.rvs(loc=3, scale=0.5, size=n)
modl = stats.poisson.rvs(mu=30, size=n)
process_int = stats.norm.rvs(loc=50, scale=20, size=n)
pipe = stats.poisson.rvs(mu=100, size=n)
modl_prod = stats.norm.rvs(loc=30, scale=5, size=n)
```

[3]:
```
# Without Stakeholder analysis we cannot do development + pipelline
# No steps with EDA or after without extraction
devel = eda + modl + modl_prod
prod = pipe
print((devel > prod).sum()/n)
# percent where development takes longer than production
```

```
0.001286
```

1.

[4]:
```
days = sh_analysis + extr + np.amax(np.column_stack((devel,prod)),1) +␣
↪process_int

print("Mean number of days for process:", days.mean())
print("Median number of days for process:", np.median(days))
```

```
Mean number of days for process: 175.01806446150718
Median number of days for process: 174.51630290967773
```

[5]:
```
print("two sided confidence interval:", np.quantile(days, [0.025, 0.975]))
```

```
two sided confidence interval: [128.04939034 224.92641197]
```

2.

[6]:
```
print("Variance of production days:", np.var(prod))
print("Mean of production days:", np.mean(prod))
print("Variance of development days:", np.var(devel))
print("Mean of development days:", np.mean(devel))
print("Variance of process integration days:", np.var(process_int))
print("Mean of process integration days:", process_int.mean())
print("Variance of stakeholder analysis days:", np.var(sh_analysis))
print("Mean of stakeholder analysis days:", sh_analysis.mean())
print("Variance of extraction days:", np.var(extr))
print("Mean of extraction days:", extr.mean())
```

```
Variance of production days: 99.96511678476399
Mean of production days: 99.995394
Variance of development days: 55.223406004007536
Mean of development days: 62.99018234378345
Variance of process integration days: 399.6367302622432
Mean of process integration days: 50.02606868266868
Variance of stakeholder analysis days: 99.91415811677727
Mean of stakeholder analysis days: 9.991634460462008
Variance of extraction days: 8.983715460453245
Mean of extraction days: 15.000476505592223
```

The variance is the highest for the Process Integration step, which can only be done after the Production Pipeline and model productionization are complete. Since it has the highest variance in days units, it contributes the most towards process time uncertainininty.

The data pipeline contributes most to the overall duration because its mean is 100 days and is almost always the bottleneck in the dev vs prod step.

[7]:
```
# So days represents our 1 million simulations of the total days for the␣
 ↪process withou delay
additional_pipe_time = stats.poisson.rvs(50,size=n)
prod_with_delay = pipe + additional_pipe_time

days_w_delay = sh_analysis + extr + np.amax(np.
 ↪column_stack((devel,prod_with_delay)),1) + process_int

length_of_delay = (days_w_delay - days)
print("Mean length of delay:", length_of_delay.mean())
print("two sided confidence interval for length of delay:", np.
 ↪quantile(length_of_delay, [0.025, 0.975]))
```

```
Mean length of delay: 50.00037118721575
two sided confidence interval for length of delay: [37. 64.]
```